Caso Semestral: "Colegio Bernardo O'Higgins – Plataforma de libro de clases digital"

Integrantes: Vicente Manriquez, Davier Ramos, Dilan Micolta

Asignatura: DESARROLLO FULLSTACK III (DSY1106).

Sección: 002D

Fecha: 09/04/2026

Índice de Contenidos

1. Contextualización del caso 3
2. Selección de tecnologías y herramientas técnicas 3
3. Selección y justificación de patrones de arquitectura 4
4. Comunicación entre microservicios 7
5. Arquitectura de microservicios y contenedores 8
6. Evaluación general del diseño (MVP) 9
7. Bibliografía 10

1. Contextualización del caso

El Colegio Bernardo O'Higgins busca implementar una plataforma digital de libro de clases que permita gestionar de manera centralizada la información académica, reemplazando el uso de registros físicos y sistemas fragmentados. El sistema debe considerar múltiples actores: docentes, estudiantes, apoderados y personal administrativo.

Los procesos clave a incorporar incluyen la matrícula de estudiantes, asignación a cursos, gestión de asignaturas y docentes, registro de asistencia, ingreso de calificaciones, generación de reportes en PDF y la comunicación entre los distintos actores. Dado el alcance del proyecto y su duración estimada de tres meses, la solución se abordará como un Producto Mínimo Viable (MVP), priorizando funcionalidades esenciales que permitan validar la propuesta tecnológica sin sobrecargar el desarrollo inicial.

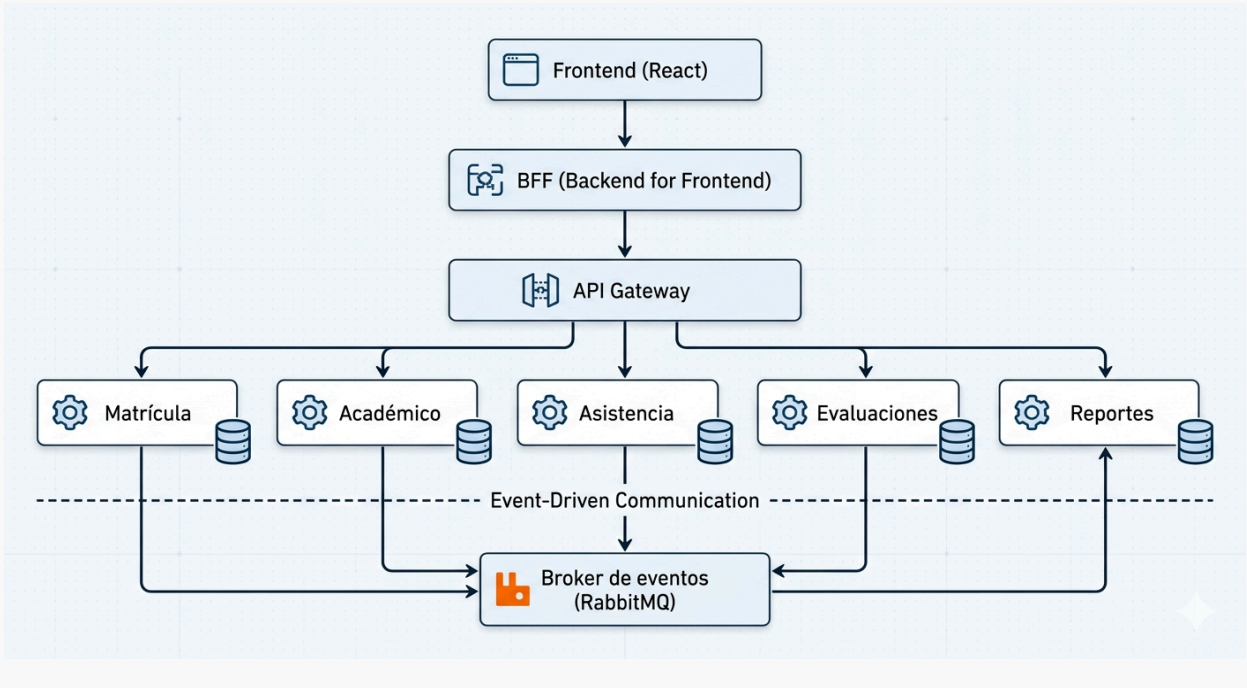
2. Selección de tecnologías y herramientas técnicas

Para el desarrollo del sistema se seleccionó un conjunto de tecnologías alineadas con los requerimientos del caso y el enfoque MVP, priorizando herramientas robustas y escalables:

- **Framework Backend:** Java 21, Spring Boot, Spring Cloud, Spring Data JPA (Hibernate) y Maven. Spring Boot permite desarrollar microservicios estructurados, mientras que Spring Cloud facilita la implementación de patrones como API Gateway. JPA permite la persistencia eficiente de datos.
- **Frontend:** React.js, Vite y Axios. React permite construir interfaces modulares empaquetadas como módulos NPM, y Vite optimiza el rendimiento en la etapa de desarrollo.

- **Base de Datos:** PostgreSQL. Es un motor relacional ideal para manejar las estructuras del colegio (estudiantes, cursos) y es plenamente compatible con JPA.
- **Mensajería Asíncrona:** RabbitMQ. Permite implementar una arquitectura basada en eventos para procesos en segundo plano.
- **Seguridad:** Spring Security y JWT (JSON Web Tokens) para controlar el acceso mediante autenticación y autorización por roles (docente, apoderado, administrador).
- **Despliegue y Orquestación:** Docker y Kubernetes.

3. Selección y justificación de patrones de arquitectura

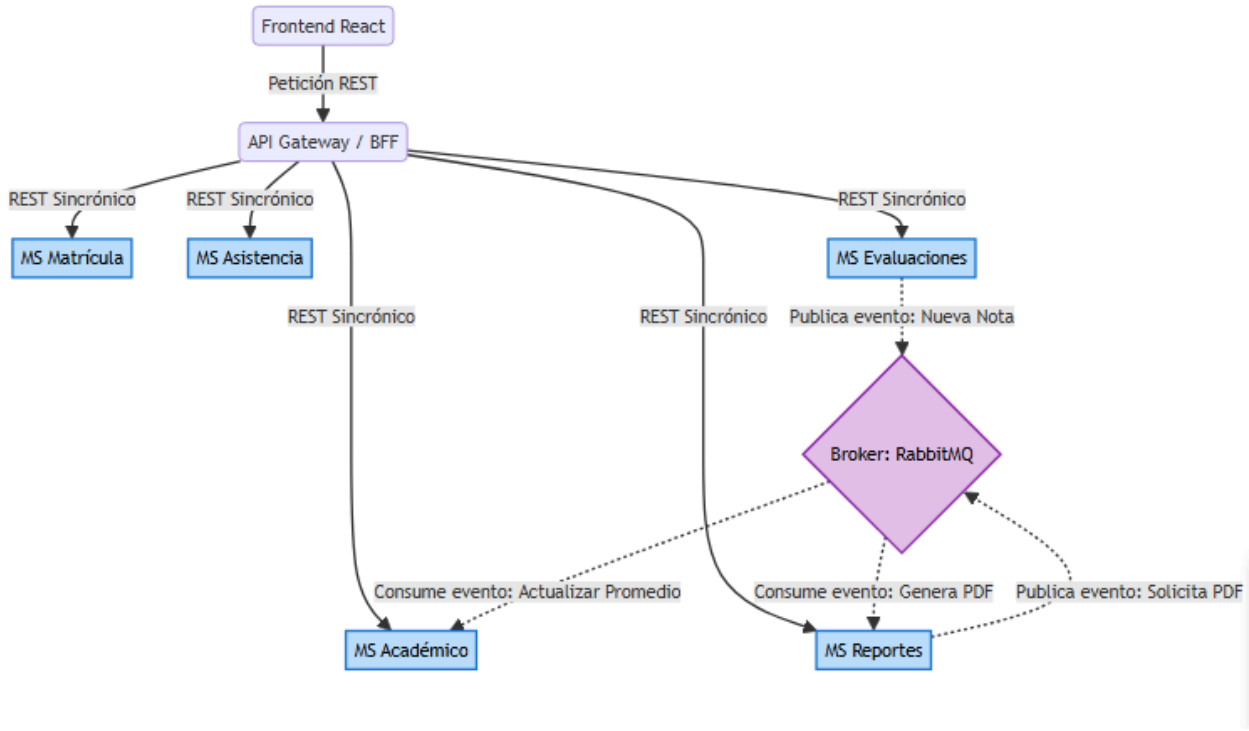


Se adopta una arquitectura basada en microservicios, utilizando un enfoque híbrido de comunicación REST y Event-Driven Architecture (EDA).

- **Microservicios:** El sistema se divide en dominios independientes (Matrícula, Académico, Asistencia, Evaluaciones, Reportes). Esto facilita la escalabilidad, el mantenimiento y el desarrollo paralelo por parte del equipo.

- **API Gateway y Backend For Frontend (BFF):** El API Gateway centraliza las solicitudes, brindando un punto único de entrada que mejora la seguridad y el control de tráfico. El BFF adapta la información específicamente para las necesidades de la interfaz en React, reduciendo llamadas innecesarias y mejorando el rendimiento.
- **Event-Driven Architecture (EDA):** Fundamental para procesos pesados como la generación de reportes PDF y notificaciones. Evita bloqueos en el sistema mediante comunicación asíncrona.
- **Repository Pattern:** Separa la lógica de negocio del acceso a la base de datos PostgreSQL, mejorando la mantenibilidad del código.
- **Factory Method:** Permite la creación flexible de objetos complejos, facilitando la futura extensión del sistema.
- **Circuit Breaker:** Previene fallos en cascada. Si el servicio de reportes cae, el Circuit Breaker asegura que el registro de asistencia siga funcionando con normalidad.

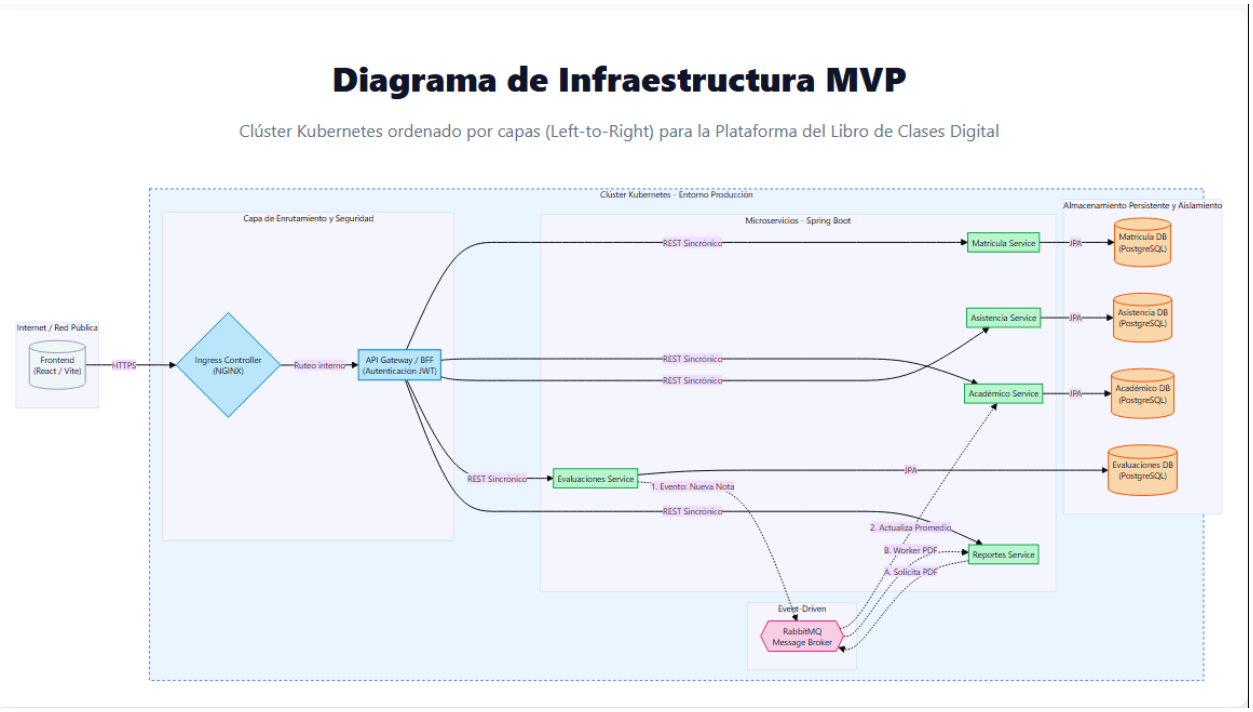
4. Comunicación entre microservicios



La plataforma utiliza dos paradigmas de comunicación para asegurar la eficiencia técnica y operativa:

- **Comunicación Sincrónica (REST):** Utilizada para interacciones que requieren respuesta inmediata, tales como consultas de datos, proceso de matrícula, registro de asistencia y anotación de calificaciones.
- **Comunicación Asíncrona (Eventos con RabbitMQ):** Utilizada para tareas que consumen tiempo. Por ejemplo, cuando un usuario solicita un reporte mediante REST, el servicio de reportes publica un evento. El PDF se genera en segundo plano y posteriormente se notifica al usuario. Esto reduce la carga en los servicios principales y permite escalar procesos críticos de forma aislada.

5. Arquitectura de microservicios y contenedores



Para el MVP se han definido los siguientes microservicios independientes: Matrícula, Académico, Asistencia, Evaluaciones y Reportes. Cada uno de estos microservicios será desplegado en un contenedor Docker independiente, asegurando portabilidad y aislamiento.

La gestión de estos contenedores se realizará mediante Kubernetes. En el contexto de este MVP, Kubernetes aporta un valor fundamental simulando un entorno productivo real:

- **Escalabilidad automática:** Permite aumentar instancias de microservicios específicos según la demanda (ej. más recursos para "Evaluaciones" a fin de semestre).

- **Alta disponibilidad y balanceo:** Distribuye el tráfico y redistribuye servicios automáticamente en caso de fallos en los nodos.

Desde la perspectiva de la seguridad, la arquitectura centraliza el acceso en el API Gateway utilizando HTTPS y validación JWT. La privacidad se garantiza mediante el control de acceso por roles y la separación de datos por servicio. Finalmente, el diseño aporta a la sostenibilidad, ya que el escalado independiente optimiza el uso de recursos computacionales, evitando mantener servidores encendidos sin uso justificado.

6. Evaluación general del diseño (MVP)

La arquitectura propuesta permite cumplir con los requerimientos del sistema de manera eficiente. Sin embargo, al tratarse de un MVP, se han identificado ciertas consideraciones:

- **Debilidades y Riesgos:** Existe una alta complejidad inicial en la configuración de Kubernetes y RabbitMQ. La gestión de datos distribuida aumenta la dificultad de depuración (debugging) y presenta riesgos potenciales de consistencia de datos si ocurren problemas en la red interna del clúster.
- **Mejoras futuras:** Para iteraciones posteriores se recomienda implementar sistemas de caché (Redis) para consultas frecuentes, monitoreo de logs centralizados (Prometheus/Grafana) y mayor automatización en los pipelines de ci/cd mediante GitHub Actions.

En **conclusión**, la selección de herramientas responde a la necesidad de construir una arquitectura moderna. Tecnologías como Spring Boot y React permiten un desarrollo ágil, mientras que Docker, Kubernetes y RabbitMQ garantizan un sistema desacoplado y resiliente. El enfoque MVP asegura la entrega de un sistema funcional dentro de los plazos establecidos, preparado para escalar hacia un entorno productivo definitivo.

7. Bibliografía

- Fowler, M. (2018). *Microservices Patterns*. Manning Publications.
- Newman, S. (2021). *Building Microservices*. O'Reilly Media.
- Oracle. (2023). *Java Persistence API Documentation*.
- React Documentation. (2024). *React.dev*.
- Richardson, C. (2020). *Microservices Patterns*. Manning Publications.